Report

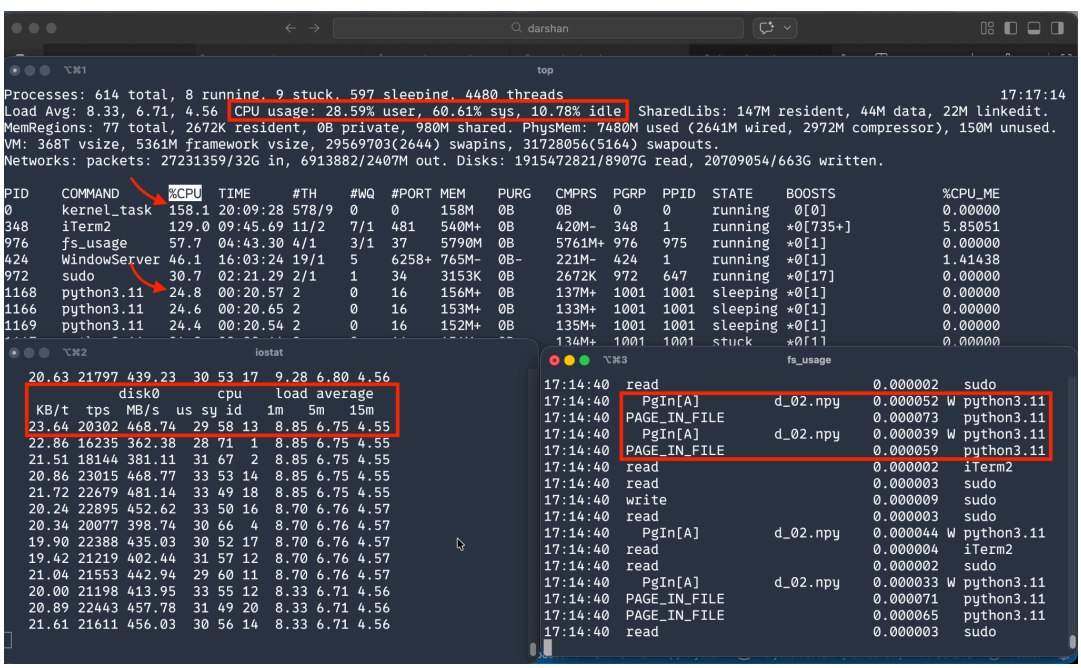
Task 1 : Understanding PyTorch DataLoader Configuration

Phase 1 - A reproducible dataset (all 0's) of ~5GiB is created using `generate_dataset.py` and 10 shards of this dataset is generated. The details of this dataset is stored in a `metadata.yaml` file in the current directory.

Phase 2 - A custom Dataset class is implemented using `synthetic_dataset.py`, where `__init__(self), __len__(self), __getitem__(self, idx)` are implemented. I have tried to cache the memmap objects so that each shard file is opened only once per worker and is re-used for all future samples, inside the dataset object.

Phase 3 - In the `benchmark.py`, I do only a single warm up pass to exclude the initialization overhead and 5 times the benchmark is run before calculating the Average I/O time, I/O Throughput and Observed scaling efficiency across all the different `num_worker` configurations.

Executive Results Summary – The DataLoader shows sub-linear I/O scaling : when we move from 1 worker to 8 workers, we obtain only ~1.6x speedup (from 4982 samples/sec to 7957 samples/sec) and scaling efficiency drops from 100% to ~20%. Further profiling with top, iostat and fs_usage commands reveal that heavy kernel activity and many page-in reads, due to memory-mapped dataset access. These page faults trigger many small disk reads handled by the operating system, making disk I/O and kernel overhead the dominant bottlenecks.



Task 2 : Darshan I/O Analysis

Key I/O Characteristics and Patterns – Across ranks, it is important to note that : the application reads same 52.58 MiB from the same 5 files on `/lcrc` mountpoint and writes roughly the same ~211-225 KiB and ~575 B to `stderr` and `stdout` respectively. These highlight that I/O volume remains identical across different rank configurations. It seems that DataLoader workers almost don't use STDIO at all (except reading 2 bytes from a file, which I suspect is reading some file metadata). The real data loading however happens through POSIX (reading dataset files) and all output happens through POSIX writes to `stderr` and `stdout`. In POSIX module, the read access pattern is completely unchanged (same # of read, open & seek operations). The write access pattern is, however, sequential and these writes are to `stderr` (most likely warning messages) and on average ~80-88 bytes are written in every write operation, which also reflects the "Access Sizes" bar graph. The remaining 2 non-sequential writes are to `stdout`. The heatmaps show the temporal read & write behaviour. Across all heatmaps and across all the ranks, a common pattern exists - a 2B STDIO read indicates the start of the data loading phase and then a dark concentrated burst of reads (52.58MiB) to load the dataset, followed by a yellow-steady state throughout the rest of training dominated by continuous small writes to `stderr`. We can also verify this by calculating the # of bytes per bin and this would match the colors on the color scale.

Q1. Does I/O scale proportionally with the number of ranks ?

Workers	Runtime	I/O Throughput	Total Bytes Read	Time spent doing I/O	Throughput Speedup (relative to w = 1)
1	244.3 s	126.61 MiB/sec	52.58 MiB	52.58 MiB / 126.61 MiB / sec ~ 0.42 sec	1x
2	298.6 s	242.81 MiB/sec	52.58 MiB	~ 0.22 sec	1.9x
4	209.8 s	507.56 MiB/sec	52.58 MiB	~ 0.10 sec	4x
8	215.1 s	474.52 MiB/sec	52.58 MiB	~ 0.11 sec	3.7x
32	228.3 s	473.50 MiB/sec	52.58 MiB	~ 0.11 sec	3.7x

The actual I/O takes < 0.5 sec for every `num_worker` configuration (< 0.2% of a 244 - 299 sec runtime), which describes the I/O cost graph, where the bars are essentially invisible. No, the I/O does not scale linearly with the number of ranks. The saturation point is `num_worker` = 4 for workload and filesystem. Adding more concurrent readers beyond this point provides no additional throughput. In fact, I feel that beyond 4 workers, worker overhead becomes large due to process spawning and IPC, while the actual useful work per worker shrinks.

Q2. What filesystem level issues might occur at a large scale ?

1. Metadata server bottleneck on shared filesystem – Metadata server that handles all filesystem metadata operations - like directory operations, file lookups, file creation and deletion can be overloaded if too many processes open lots of files and each “open” may invoke further multiple metadata operations like file lookup, permission checks, etc. So, the MDS might see 10s-100s of thousands of metadata RPCs within a short burst at job start, which can also lead to network bandwidth contention.
2. Filesystem journaling overhead – Every metadata operation (creating a file, changing permissions, updating timestamps) and every write operation generates journal entries which are maintained in a journal by the filesystem to protect against crashes and data corruption. The journal itself is stored on disk & writing to it is serialized. At a large scale, hundreds of processes generating metadata operations simultaneously create a stream of journal writes that the filesystem must handle sequentially. This can become very problematic for write-heavy workloads (like simulations).
3. Access pattern mismatch with filesystem optimization – Parallel filesystems are designed for large sequential I/O like large reads and writes in order. This matches traditional HPC simulation workloads but ML Training workloads have a different pattern. A DataLoader with `shuffle` = True, reads samples in a random order across the dataset. If the dataset is a single large file, the filesystem sees random seeks rather than sequential reads. Similarly, disk I/O scheduler that collect pending read requests & reorder them in order to minimize disk’s read head movement cannot do so.

Q3. What changes to data loading strategy could improve scalability ?

1. Accelerating large ML training pipelines – Move expensive data pre-processing such as transformations applied on raw data outside (like feature extraction) main training loop & write the results into sharded data files. This can help avoid redoing the same work each epoch, not to mention the wasted CPU cost every epoch. Before training run, these shards can be placed on physical storage attached to compute nodes, making training faster and avoiding overloading the filesystem.
2. Tuning different knobs at model-level & system-level – `num_workers` and `persistent_workers=True` which ensures that those workers stay alive across epochs, avoiding process startup overhead. Binding worker processes to a set of cores is helpful so that they don’t interfere with the main training process. Using NUMA aware placement for worker processes on multi-socket machines is helpful as it reduces remote memory accesses, reduces cross-socket traffic and as a result, each worker finishes data prep work faster.
3. Combining many small files into larger shards – As discussed, in large HPC systems, MDS can be overloaded when we have lots of accesses to lots of small files. A better approach would be to consolidate many small files into a large file shard. This file shard can for example contain many training samples, and we can read these samples sequentially in DataLoader. However, if DataLoader randomly accesses samples within a shard, then the benefit of sharding is lost. An example file format is `.tar`.